RAJALAKSHMI ENGINEERING COLLEGE
RAJALAKSHMI NAGAR, THANDALAM – 602 105

RAJALAKSHMI ENGINEERING COLLEGE

CS23334 – Fundamentals of Data Science Lab

Laboratory Record Note Book

Name : SALLY MADHULIKA

Year / Branch / Section : II - CSE - H

University Register No. : 211625070163 3

College Roll No. : 250701633

Semester : III

Academic Year : 2026 - 2027

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

CS23334 – Fundamentals of Data Science Lab

NAME	SALLY MADHULIKA
ROLL NO	2116250701633
YEAR	II
SEC	H

RAJALAKSHMI ENGINEERING COLLEGE
An Autonomous Institution

BONAFIDE CERTIFICATE

Name: SALLY MADHULIKA . M . R

Academic Year: 26-27 **Semester:** III **Branch:** CSE

Register No. 2116250701633

Certified that this is the bonafide record of work done by the above student in the FUNDAMENTALS OF DATA SCIENCE *Laboratory during the academic year 2026 – 2027.*

3/11/26

Signature of Faculty in-charge

Submitted for the Practical Examination held on 06-11-2026

Internal Examiner

External Examiner

INDEX

Sl.NO	Experiment	Date	GitHub
1.a	Analyze the trend of data science job postings over the last decade for the given data on the number of data science job postings each year. Use pandas for data manipulation and matplotlib /seaborn for visualization.	29/07/26	
1.b	Analyze and visualize the distribution of various data science roles (Data Analyst, Data Engineer, Data Scientist, etc.) from a dataset.	29/07/26	
1.c	Conduct an experiment to differentiate Structured , Un-structured and Semi structured data based on data sets given.	05/08/26	
1.d	Conduct an experiment to encrypt and decrypt given sensitive data Use the cryptography library fernet.	05/08/26	
2	Upload and Analyze the data set given in csv format and perform data preprocessing and visualization.	12/08/26	
3.a	Conduct an experiment to show data visualization using line plot	19/08/26	
3.b	Conduct an experiment to show data visualization using bar chart	19/08/26	
3.c	Conduct an experiment to show data visualization using pie chart	19/08/26	
4	To Count the frequency of occurrence of a word in a body of text is often needed during text processing.	26/08/26	
5	Data Collection and Initial Exploration	16/09/26	
6	Time Series Analysis of Website Traffic	16/09/26	

7	Random Sampling and Sampling Distribution	23/09/26	
8	Conduct Z test for the Data given using Python	23/09/26	
9	Experiment to understand Matplotlib library use cases in Data Science through visualization.	23/09/26	
10	Experiment to understand array function in Data science.	23/09/26	
11	Experiment to understand pandas library use cases in Data science.	30/09/26	
12	Experiment to detect outliers in a given data set.	30/09/26	
13	Experiment to handle missing values and inappropriate data in a given data set.	30/09/26	
14	Experiment to understand feature scaling	30/09/26	
15	Experiment to understand the data preprocessing in Data science	30/09/26	
16	Experiment to understand EDA-Quantitative and Qualitative analysis.	07/10/26	
17	Experiment to understand Linear Regression for a given data set.	07/10/26	
18	Experiment to understand KNN algorithm for a given data set.	07/10/26	
19	Experiment to understand Logistic Regression for a given data set.	07/10/26	
20	Experiment to understand K-means clustering algorithm for a given data set.	07/10/26	

Exp No:1.a Analyse the trend of data science job postings over the last

Decade

CODE

```
import pandas as pd

import matplotlib.pyplot as plt

data = {'Year': list(range(2010, 2021)),

'Job Postings': [150, 300, 450, 600, 800, 1200, 1600, 2100, 2700, 3400, 4200]}

df = pd.DataFrame(data)

plt.plot(df['Year'], df['Job Postings'], marker='o')

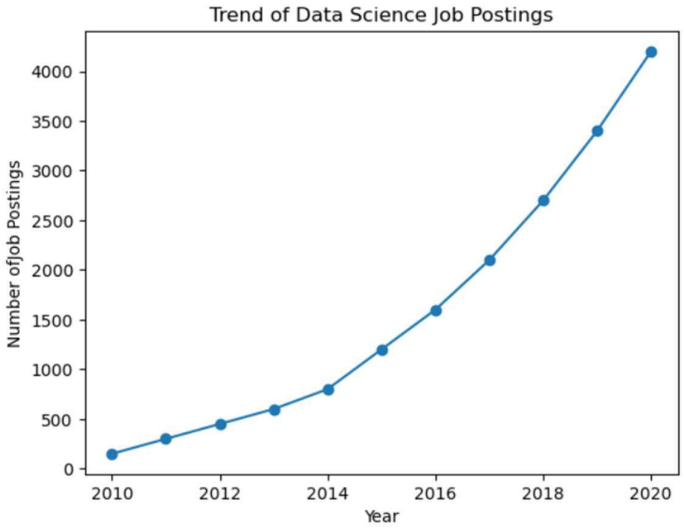
plt.title('Trend of Data Science Job Postings')

plt.xlabel('Year')

plt.ylabel('Number ofJob Postings')

plt.show()
```

output



Result: the trend of data science job postings for the last decade has been analyzed

Exp No:1.b Analyze and visualize the distribution of various data science roles (Data Analyst, Data Engineer, Data Scientist, etc.) from a dataset.

Code

```
roles = ['Data Analyst', 'Data Engineer', 'Data Scientist', 'ML Engineer',

'Business Analyst']

counts = [300, 500, 450, 200, 150]

plt.bar(roles, counts)

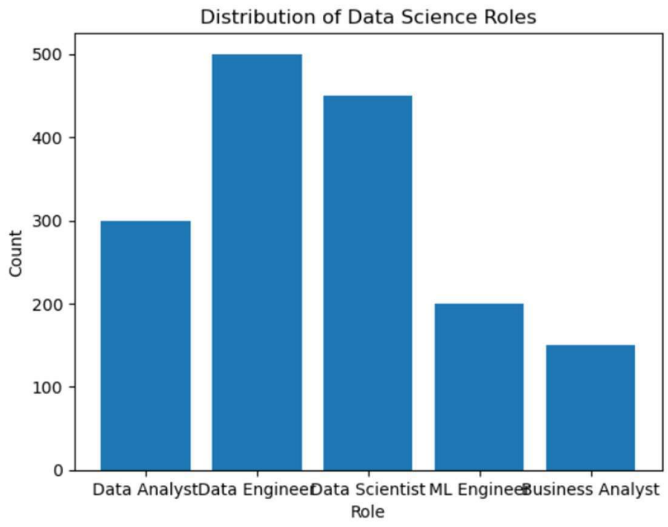
plt.title('Distribution of Data Science Roles')

plt.xlabel('Role')

plt.ylabel('Count')

plt.show()
```

output



Result: the distribution of various data science roles has been analysed

Exp No:1.c Conduct an experiment to differentiate Structured , Un-structured and Semi-structured data based on data sets given.

CODE:

```
structured_data = pd.DataFrame({

'ID': [1, 2, 3],

'Name': ['Sally Madhulika', 'Sanjita', 'Sangavi'],

'Age': [25, 30, 35]

})

print("Structured Data:\n", structured_data)

unstructured_data = "my name is Sally Madhulika”

print("\nUnstructured Data:\n", unstructured_data)

semi_structured_data = {'ID': 1, 'Name': 'Sally Madhulika’,
'Attributes': {'Height': 170, 'Weight': 60}}

print("\nSemi-structured Data:\n", semi_structured_data)
```

OUTPUT:

```
Structured Data:
    ID             Name  Age
0   1  Sally Madhulika   25
1   2          Sanjita   30
2   3          Sangavi   35

Unstructured Data:
 My name is Sally Madhulika

Semi-structured Data:
 {'ID': 1, 'Name': 'Sally Madhulika', 'Attributes': {'Height': 170, 'Weight': 60}}
```

RESULT: hence, Structured , Un-structured and Semi-structured data are differentiated.

Exp No:1.d Conduct an experiment to encrypt and decrypt given sensitive data.

CODE:

```
from cryptography.fernet import Fernet

key = Fernet.generate_key()

f = Fernet(key)

token = f.encrypt(b"Rajalakshmi Engineering College")

token

b'...'

f.decrypt(token)

b'Rajalakshmi Engineering College'

key = Fernet.generate_key()

cipher_suite = Fernet(key)

plain_text = b"Rajalakshmi Engineering College."

cipher_text = cipher_suite.encrypt(plain_text)

decrypted_text = cipher_suite.decrypt(cipher_text)

print("Original Data:", plain_text)

print("Encrypted Data:", cipher_text)

print("Decrypted Data:", decrypted_text)
```

OUTPUT:

```
Original Data: b'Rajalakshmi Engineering College.'
Encrypted Data: b'gAAAAABqeWlqKrpskFqHBbhirNcJIdQBRWYG1yD8Mub-tmK5MoReCIb12-ME25PIv-cZIxPyhUfJvm8C9HM1tHAi2lWGyoXYLHx9H7GSaOP4DPfpnZMnYwfT4btsJr6w9tA4omG_1egq'
Decrypted Data: b'Rajalakshmi Engineering College.'
```

RESULT: Hence given sensitive data has been encrypted and decrypted.

Exp No:2 Upload and Analyze the data set given in csv format and perform data preprocessing and visualization.

CODE:

```
import pandas as pd

import numpy as np

import matplotlib.pyplot as plt

df = pd.DataFrame({

   'Date': ['01-01-2023', '02-01-2023','03-01-2023','04-01-2023','05-01-2023',],

   'Product': ['Product A', 'Product B','Product A', 'Product C','Product B',],

   'Sales': [200,150,220,300,180,],

   'Quantity': [4,3,5,6,4,],

   'Region': ['North','South','North','East','West',]

})

print(df.head())

print(df.isnull().sum())

df['Sales'] = df['Sales'].fillna(df['Sales'].mean())

df.dropna(subset=['Product', 'Quantity', 'Region'], inplace=True)

print(df.describe())

product_summary = df.groupby('Product').agg({

   'Sales': 'sum',

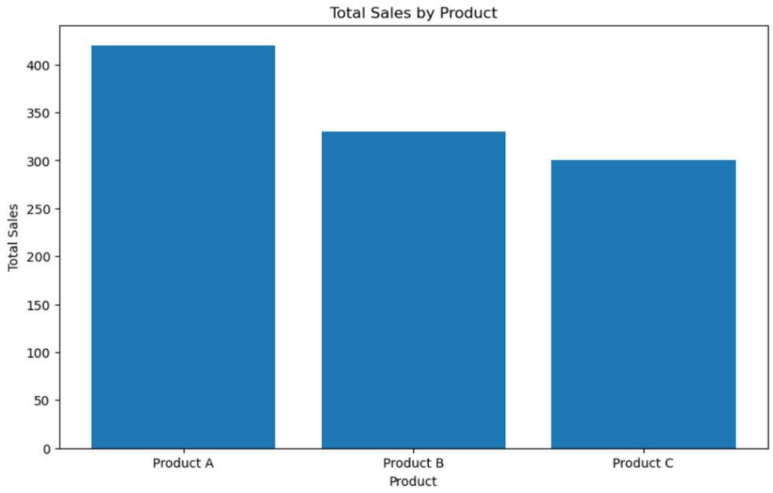
   'Quantity': 'sum'
```

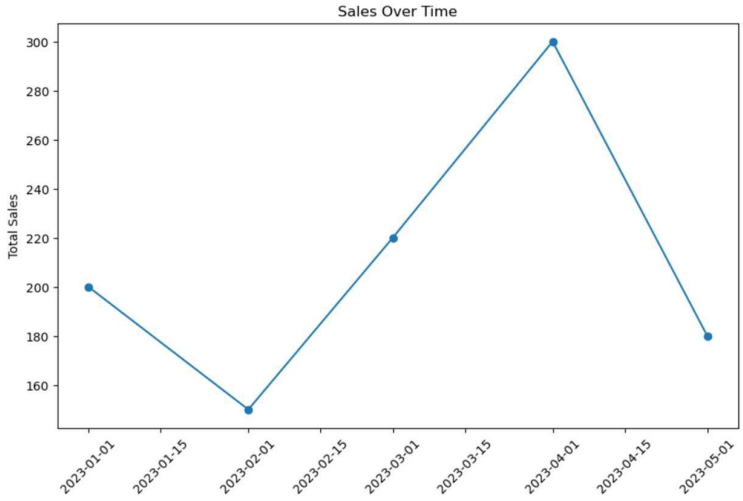
```
}).reset_index()
print(product_summary)
plt.figure(figsize=(10, 6))
plt.bar(product_summary['Product'], product_summary['Sales'])
plt.xlabel('Product')
plt.ylabel('Total Sales')
plt.title('Total Sales by Product')
plt.show()
df['Date'] = pd.to_datetime(df['Date'])
sales_over_time = df.groupby('Date')['Sales'].sum().reset_index()
plt.figure(figsize=(10, 6))
plt.plot(sales_over_time['Date'], sales_over_time['Sales'],
marker='o')
plt.xlabel('Date')
plt.ylabel('Total Sales')
plt.title('Sales Over Time')
plt.xticks(rotation=45)
plt.show()
pivot_table = df.pivot_table(
    values='Sales',
    index='Region',
    columns='Product',
    aggfunc='sum',
```

```
    fill_value=0
)
print(pivot_table)
correlation_matrix = df.corr(numeric_only=True)
print(correlation_matrix)
plt.figure(figsize=(8, 6))
plt.imshow(correlation_matrix, cmap='coolwarm')
plt.colorbar()
plt.xticks(range(len(correlation_matrix.columns)),
correlation_matrix.columns)
plt.yticks(range(len(correlation_matrix.columns)),
correlation_matrix.columns)
plt.title('Correlation Matrix')
plt.show()
```

OUTPUT

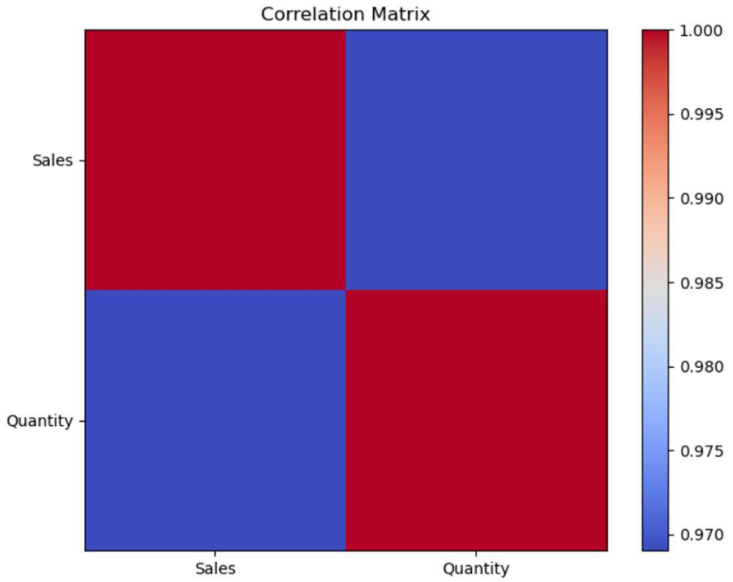
```
         Date    Product  Sales  Quantity Region
0  01-01-2023  Product A    200         4  North
1  02-01-2023  Product B    150         3  South
2  03-01-2023  Product A    220         5  North
3  04-01-2023  Product C    300         6   East
4  05-01-2023  Product B    180         4   West
Date        0
Product     0
Sales       0
Quantity    0
Region      0
dtype: int64
            Sales  Quantity
count    5.000000  5.000000
mean   210.000000  4.400000
std     56.568542  1.140175
min    150.000000  3.000000
25%    180.000000  4.000000
50%    200.000000  4.000000
75%    220.000000  5.000000
max    300.000000  6.000000
     Product  Sales  Quantity
0  Product A    420         9
1  Product B    330         7
2  Product C    300         6
```





Product	Product A	Product B	Product C
Region			
East	0	0	300
North	420	0	0
South	0	150	0
West	0	180	0

	Sales	Quantity
Sales	1.000000	0.969021
Quantity	0.969021	1.000000



RESULT: Hence data preprocessing and visualization has been performed on the data set given in csv format.

Exp No:3 Experiment to handle missing values and inappropriate data in a given data set

CODE

```
import numpy as np

import pandas as pd

df = pd.DataFrame({

   'Country':
['India','Morocco','Germany','Morocco','Germany','India','Morocco','I
ndia','Germany','India'],

   'Age': [44, 27, 30, 38, 40, 35, np.nan, 48, 50, 37],

   'Salary': [72000, 48000, 54000, 61000, np.nan, 58000, 52000,
79000, 83000, 67000],

   'Purchased': ['No', 'Yes', 'No', 'No', 'Yes', 'Yes',

            'No', 'Yes', 'No', 'Yes']

})

print("Original Dataset:")

print(df)

print("\n\nDataFrame Information:")

print(df.info())

print("\n\nCountry Mode:")

print(df.Country.mode())

print("\n\nFirst Mode Value:")

print(df.Country.mode()[0])

df['Country'] = df['Country'].fillna(df.Country.mode()[0])
```

```
df['Age'] = df['Age'].fillna(df.Age.median())
df['Salary'] = df['Salary'].fillna(round(df.Salary.mean()))
print("\n\nDataset After Filling Missing Values:")
print(df)
print("\n\nOne Hot Encoding:")
print(pd.get_dummies(df.Country))
updated_dataset = pd.concat(
    [pd.get_dummies(df.Country), df.iloc[:, [1, 2, 3]]],
    axis=1
)
print("\n\nUpdated Dataset:")
print(updated_dataset)
print("\n\nDataFrame Information After Preprocessing:")
print(df.info())
updated_dataset['Purchased'] =
updated_dataset['Purchased'].replace(
    ['No', 'Yes'], [0, 1]
)
print("\n\nFinal Dataset:")
print(updated_dataset)
```

OUTPUT

	CustomerID	Age_Group	Rating(1-5)	Hotel	FoodPreference	Bill	NoOfPax	EstimatedSalary	Age_Group.1
0	1	20-25	4	Ibis	veg	1300	2	40000	20-25
1	2	30-35	5	LemonTree	Non-Veg	2000	3	59000	30-35
2	3	25-30	6	RedFox	Veg	1322	2	30000	25-30
3	4	20-25	-1	LemonTree	Veg	1234	2	120000	20-25
4	5	35+	3	Ibis	Vegetarian	989	2	45000	35+
5	6	35+	3	Ibys	Non-Veg	1909	2	122220	35+
6	7	35+	4	RedFox	Vegetarian	1000	-1	21122	35+
7	8	20-25	7	LemonTree	Veg	2999	-10	345673	20-25
8	9	25-30	2	Ibis	Non-Veg	3456	3	-99999	25-30
9	9	25-30	2	Ibis	Non-Veg	3456	3	-99999	25-30
10	10	30-35	5	RedFox	non-Veg	-6755	4	87777	30-35

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 11 entries, 0 to 10
Data columns (total 9 columns):
 #   Column            Non-Null Count  Dtype
---  ------            --------------  -----
 0   CustomerID        11 non-null     int64
 1   Age_Group         11 non-null     object
 2   Rating(1-5)       11 non-null     int64
 3   Hotel             11 non-null     object
 4   FoodPreference    11 non-null     object
 5   Bill              11 non-null     int64
 6   NoOfPax           11 non-null     int64
 7   EstimatedSalary   11 non-null     int64
 8   Age_Group.1       11 non-null     object
dtypes: int64(5), object(4)
memory usage: 924.0+ bytes
```

	CustomerID	Age_Group	Rating(1-5)	Hotel	FoodPreference	Bill	NoOfPax	EstimatedSalary
0	1	20-25	4	Ibis	veg	1300	2	40000
1	2	30-35	5	LemonTree	Non-Veg	2000	3	59000
2	3	25-30	6	RedFox	Veg	1322	2	30000
3	4	20-25	-1	LemonTree	Veg	1234	2	120000
4	5	35+	3	Ibis	Vegetarian	989	2	45000
5	6	35+	3	Ibys	Non-Veg	1909	2	122220
6	7	35+	4	RedFox	Vegetarian	1000	-1	21122
7	8	20-25	7	LemonTree	Veg	2999	-10	345673
8	9	25-30	2	Ibis	Non-Veg	3456	3	-99999
9	10	30-35	5	RedFox	non-Veg	-6755	4	87777

	CustomerID	Age_Group	Rating(1-5)	Hotel	FoodPreference	Bill	NoOfPax	EstimatedSalary
0	1.0	20-25	4.0	Ibis	veg	1300.0	2	40000.0
1	2.0	30-35	5.0	LemonTree	Non-Veg	2000.0	3	59000.0
2	3.0	25-30	NaN	RedFox	Veg	1322.0	2	30000.0
3	4.0	20-25	NaN	LemonTree	Veg	1234.0	2	120000.0
4	5.0	35+	3.0	Ibis	Vegetarian	989.0	2	45000.0
5	6.0	35+	3.0	Ibys	Non-Veg	1909.0	2	122220.0
6	7.0	35+	4.0	RedFox	Vegetarian	1000.0	-1	21122.0
7	8.0	20-25	NaN	LemonTree	Veg	2999.0	-10	345673.0
8	9.0	25-30	2.0	Ibis	Non-Veg	3456.0	3	NaN
9	10.0	30-35	5.0	RedFox	non-Veg	NaN	4	87777.0

	CustomerID	Age_Group	Rating(1-5)	Hotel	FoodPreference	Bill	NoOfPax	EstimatedSalary
0	1.0	20-25	4.0	Ibis	Veg	1300.0	2.0	40000.0
1	2.0	30-35	5.0	LemonTree	Non-Veg	2000.0	3.0	59000.0
2	3.0	25-30	4.0	RedFox	Veg	1322.0	2.0	30000.0
3	4.0	20-25	4.0	LemonTree	Veg	1234.0	2.0	120000.0
4	5.0	35+	3.0	Ibis	Veg	989.0	2.0	45000.0
5	6.0	35+	3.0	Ibis	Non-Veg	1909.0	2.0	122220.0
6	7.0	35+	4.0	RedFox	Veg	1000.0	2.0	21122.0
7	8.0	20-25	4.0	LemonTree	Veg	2999.0	2.0	345673.0
8	9.0	25-30	2.0	Ibis	Non-Veg	3456.0	3.0	96755.0
9	10.0	30-35	5.0	RedFox	Non-Veg	1801.0	4.0	87777.0

RESULT : hence missing values and inappropriate data in the given data set was studied and handled.

Exp No:4 Experiment to understand the data preprocessing in Data science

CODE

```
import numpy as np

import pandas as pd

df = pd.DataFrame({

   'Country':
['India','Morocco','Germany','Morocco','Germany','India','Morocco','I
ndia','Germany','India'],

   'Age': [44, 27, 30, 38, 40, 35, np.nan, 48, 50, 37],

   'Salary': [72000, 48000, 54000, 61000, np.nan, 58000, 52000,
79000, 83000, 67000],

   'Purchased': ['No', 'Yes', 'No', 'No', 'Yes', 'Yes',

            'No', 'Yes', 'No', 'Yes']

})

print("Original Dataset:")

print(df)

print("\n\nDataFrame Information:")

print(df.info())

print("\n\nCountry Mode:")

print(df.Country.mode())

print("\n\nFirst Mode Value:")

print(df.Country.mode()[0])

df['Country'] = df['Country'].fillna(df.Country.mode()[0])
```

```
df['Age'] = df['Age'].fillna(df.Age.median())
df['Salary'] = df['Salary'].fillna(round(df.Salary.mean()))
print("\n\nDataset After Filling Missing Values:")
print(df)
print("\n\nOne Hot Encoding:")
print(pd.get_dummies(df.Country))
updated_dataset = pd.concat(
   [pd.get_dummies(df.Country), df.iloc[:, [1, 2, 3]]],
   axis=1
)
print("\n\nUpdated Dataset:")
print(updated_dataset)
print("\n\nDataFrame Information After Preprocessing:")
print(df.info())
updated_dataset['Purchased'] =
updated_dataset['Purchased'].replace(
   ['No', 'Yes'], [0, 1]
)
print("\n\nFinal Dataset:")
print(updated_dataset)
```

OUTPUT

	Country	Age	Salary	Purchased
0	France	44.0	72000.0	No
1	Spain	27.0	48000.0	Yes
2	Germany	30.0	54000.0	No
3	Spain	38.0	61000.0	No
4	Germany	40.0	NaN	Yes
5	France	35.0	58000.0	Yes
6	Spain	NaN	52000.0	No
7	France	48.0	79000.0	Yes
8	NaN	50.0	83000.0	No
9	France	37.0	67000.0	Yes

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 10 entries, 0 to 9
Data columns (total 4 columns):
 #   Column     Non-Null Count  Dtype  
---  ------     --------------  -----  
 0   Country    9 non-null      object 
 1   Age        9 non-null      float64
 2   Salary     9 non-null      float64
 3   Purchased  10 non-null     object 
dtypes: float64(2), object(2)
memory usage: 448.0+ bytes
```

	Country	Age	Salary	Purchased
0	France	44.0	72000.0	No
1	Spain	27.0	48000.0	Yes
2	Germany	30.0	54000.0	No
3	Spain	38.0	61000.0	No
4	Germany	40.0	63778.0	Yes
5	France	35.0	58000.0	Yes
6	Spain	38.0	52000.0	No
7	France	48.0	79000.0	Yes
8	France	50.0	83000.0	No

	France	Germany	Spain	Age	Salary	Purchased
0	True	False	False	44.0	72000.0	No
1	False	False	True	27.0	48000.0	Yes
2	False	True	False	30.0	54000.0	No
3	False	False	True	38.0	61000.0	No
4	False	True	False	40.0	63778.0	Yes
5	True	False	False	35.0	58000.0	Yes
6	False	False	True	38.0	52000.0	No
7	True	False	False	48.0	79000.0	Yes
8	True	False	False	50.0	83000.0	No

	France	Germany	Spain	Age	Salary	Purchased
0	True	False	False	44.0	72000.0	0
1	False	False	True	27.0	48000.0	1
2	False	True	False	30.0	54000.0	0
3	False	False	True	38.0	61000.0	0
4	False	True	False	40.0	63778.0	1
5	True	False	False	35.0	58000.0	1
6	False	False	True	38.0	52000.0	0
7	True	False	False	48.0	79000.0	1
8	True	False	False	50.0	83000.0	0

RESULT hence data preprocessing in Data science has been studied successfully.

Exp No:5 Experiment to detect outliers in a given data set.

CODE

```
import numpy as np
array=np.random.randint(1,100,16)
array.mean()
np.percentile(array,25)
np.percentile(array,50)
np.percentile(array,75)
np.percentile(array,100)
def outDetection(array):
sorted(array)
Q1,Q3=np.percentile(array,[25,75])
IQR=Q3-Q1
lr=Q1-(1.5*IQR)
ur=Q3+(1.5*IQR)
return lr,ur
lr,ur=outDetection(array)
lr,ur
import seaborn as sns
%matplotlib inline
sns.displot(array)
sns.distplot(array)
new_array=array[(array>lr) & (array<ur)]
```

```
sns.displot(new_array)

lr1,ur1=outDetection(new_array)

lr1,ur1

final_array=new_array[(new_array>lr1) & (new_array<ur1)]

final_array
```

OUTPUT

```
array([27, 50, 44, 6, 58, 61, 23, 86, 67, 20, 75, 7, 79, 61, 90, 54])

50.5

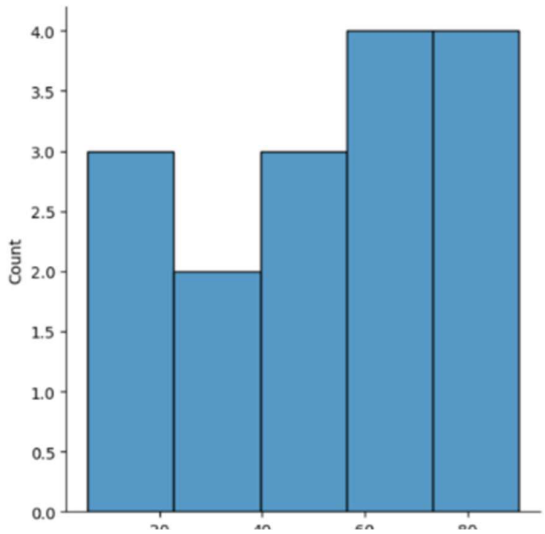
26.0

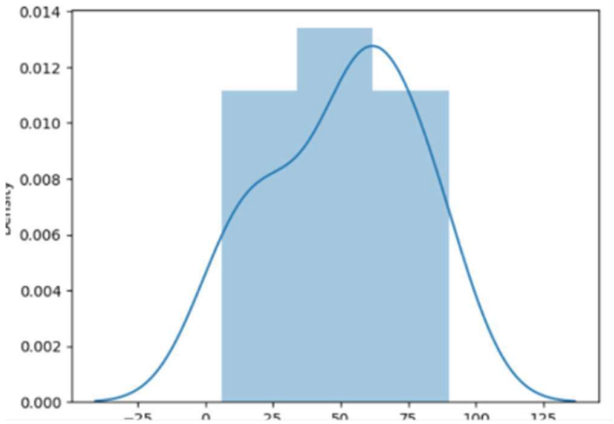
56.0

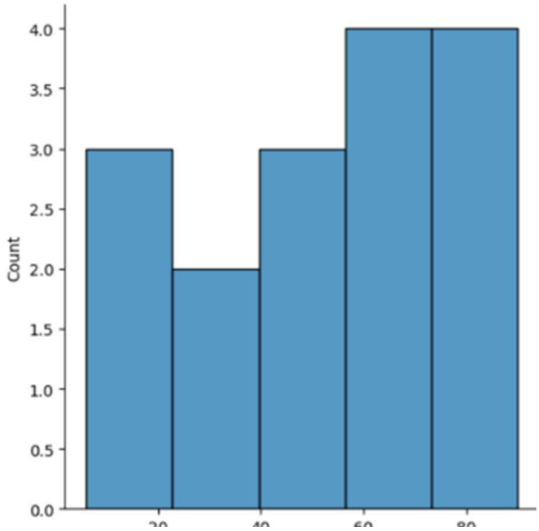
69.0

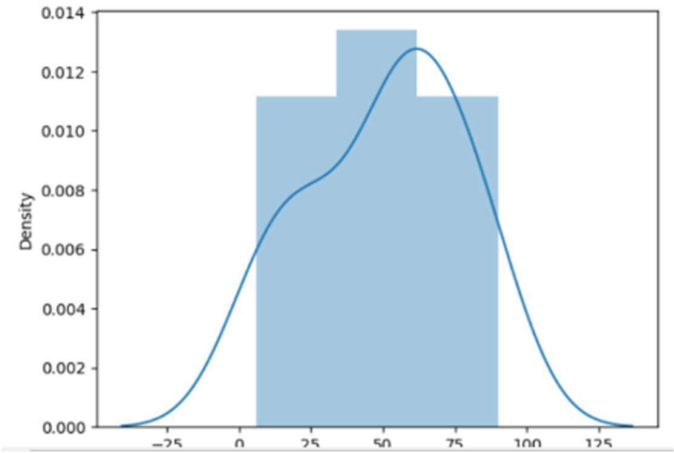
(-38.5, 133.5)

array([27, 50, 44, 6, 58, 61, 23, 86, 67, 20, 75, 7, 79, 61, 90, 54])
```









RESULT hence the outliers detected in a given data set.

Exp No:6 Experiment to understand feature scaling.

CODE

```
import numpy as np

import pandas as pd

from sklearn.impute import SimpleImputer

from sklearn.preprocessing import OneHotEncoder

from sklearn.preprocessing import StandardScaler

from sklearn.preprocessing import MinMaxScaler

df=pd.DataFrame({

'Country':['USA','France','USA','China','France','USA','China',np.nan,'France','USA'],

   'Age':[22,34,27,np.nan,45,38,31,50,29,41],
'Salary':[35000,60000,48000,72000,np.nan,55000,68000,80000,52000,64000],

   'Purchased':['No','Yes','No','Yes','No','Yes','No','Yes','No','Yes']

})

print("Original Dataset:")

print(df)

print("\n\nDataFrame Information:")

print(df.info())

df['Country']=df['Country'].fillna(df['Country'].mode()[0])

features=df.iloc[:,:-1].values

label=df.iloc[:,-1].values
```

```
age=SimpleImputer(strategy="mean",missing_values=np.nan)
Salary=SimpleImputer(strategy="mean",missing_values=np.nan)
age.fit(features[:,[1]])
Salary.fit(features[:,[2]])
features[:,[1]]=age.transform(features[:,[1]])
features[:,[2]]=Salary.transform(features[:,[2]])
print("\n\nAfter Filling Missing Values:")
print(features)
oh=OneHotEncoder(sparse_output=False)
Country=oh.fit_transform(features[:,[0]])
print("\n\nOne Hot Encoding:")
print(Country)
final_set=np.concatenate((Country,features[:,[1,2]]),axis=1)
print("\n\nFinal Set:")
print(final_set)
sc=StandardScaler()
sc.fit(final_set)
feat_standard_scaler=sc.transform(final_set)
print("\n\nStandard Scaler:")
print(feat_standard_scaler)
mms=MinMaxScaler(feature_range=(0,1))
mms.fit(final_set)
feat_minmax_scaler=mms.transform(final_set)
```

```
print("\n\nMin Max Scaler:")
print(feat_minmax_scaler)
```

OUTPUT

	Country	Age	Salary	Purchased
0	France	44.0	72000.0	No
1	Spain	27.0	48000.0	Yes
2	Germany	30.0	54000.0	No
3	Spain	38.0	61000.0	No
4	Germany	40.0	NaN	Yes
5	France	35.0	58000.0	Yes
6	Spain	NaN	52000.0	No
7	France	48.0	79000.0	Yes
8	NaN	50.0	83000.0	No
9	France	37.0	67000.0	Yes

```
array([['France', 44.0, 72000.0],
       ['Spain', 27.0, 48000.0],
       ['Germany', 30.0, 54000.0],
       ['Spain', 38.0, 61000.0],
       ['Germany', 40.0, 63777.77777777778],
       ['France', 35.0, 58000.0],
       ['Spain', 38.77777777777778, 52000.0],
       ['France', 48.0, 79000.0],
       ['France', 50.0, 83000.0],
       ['France', 37.0, 67000.0]], dtype=object)
```

```
array([[1.0, 0.0, 0.0, 44.0, 72000.0],
       [0.0, 0.0, 1.0, 27.0, 48000.0],
       [0.0, 1.0, 0.0, 30.0, 54000.0],
       [0.0, 0.0, 1.0, 38.0, 61000.0],
       [0.0, 1.0, 0.0, 40.0, 63777.77777777778],
       [1.0, 0.0, 0.0, 35.0, 58000.0],
       [0.0, 0.0, 1.0, 38.77777777777778, 52000.0],
       [1.0, 0.0, 0.0, 48.0, 79000.0],
       [1.0, 0.0, 0.0, 50.0, 83000.0],
       [1.0, 0.0, 0.0, 37.0, 67000.0]], dtype=object)
```

```
array([[1.        , 0.        , 0.        , 0.73913043, 0.68571429],
       [0.        , 0.        , 1.        , 0.        , 0.        ],
       [0.        , 1.        , 0.        , 0.13043478, 0.17142857],
       [0.        , 0.        , 1.        , 0.47826087, 0.37142857],
       [0.        , 1.        , 0.        , 0.56521739, 0.45079365],
       [1.        , 0.        , 0.        , 0.34782609, 0.28571429],
       [0.        , 0.        , 1.        , 0.51207729, 0.11428571],
       [1.        , 0.        , 0.        , 0.91304348, 0.88571429],
       [1.        , 0.        , 0.        , 1.        , 1.        ],
       [1.        , 0.        , 0.        , 0.43478261, 0.54285714]])
```

RESULT Hence the feature scaling has been studied successfully.

Exp 7 - EXPERIMENT TO UNDERSTAND EDA- Quantitative & Qualitative analysis

CODE

```
import seaborn as sns

import pandas as pd

import numpy as np

import matplotlib.pyplot as plt


%matplotlib inline

tips=sns.load_dataset('tips')

tips.head()


sns.displot(tips.total_bill,kde=True)

sns.displot(tips.total_bill,kde=False)

sns.jointplot(x=tips.tip,y=tips.total_bill)

sns.jointplot(x=tips.tip,y=tips.total_bill,kind="reg")

sns.jointplot(x=tips.tip,y=tips.total_bill,kind="hex")

sns.pairplot(tips)


tips.time.value_counts()

sns.pairplot(tips,hue='time')

sns.pairplot(tips,hue='day')


sns.heatmap(tips.corr(numeric_only=True),annot=True)

sns.boxplot(tips.total_bill)
```

```
sns.boxplot(tips.tip)

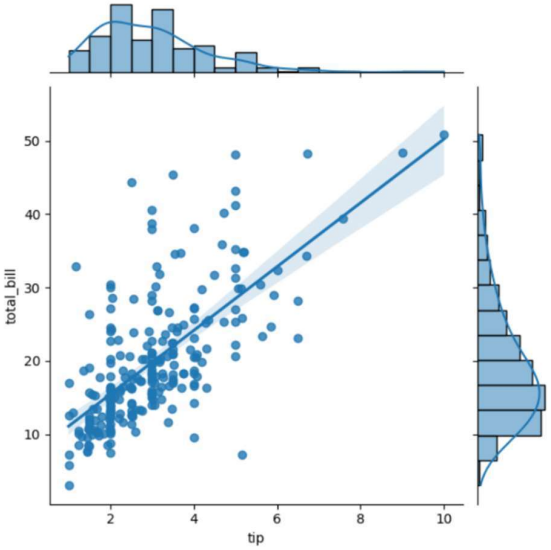
sns.countplot(tips.day)

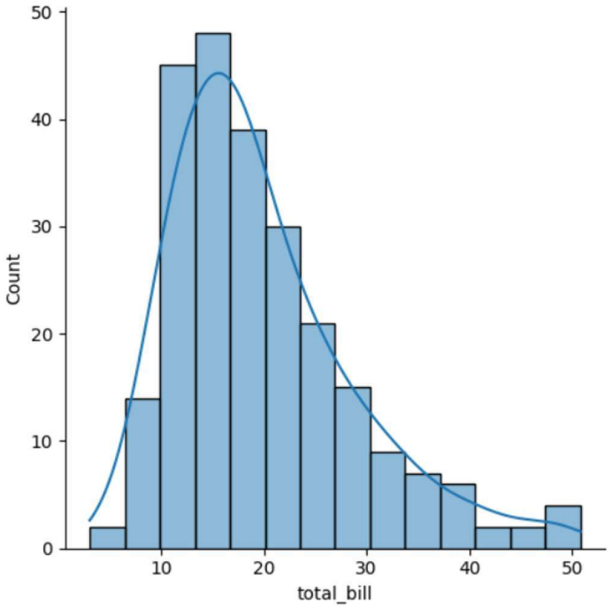
sns.countplot(tips.sex)

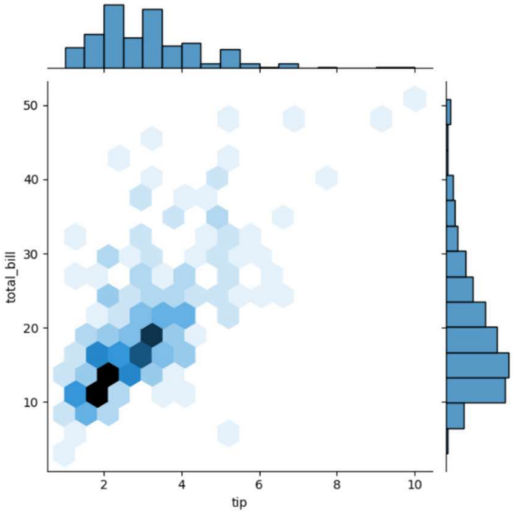

tips.sex.value_counts().plot(kind='pie')

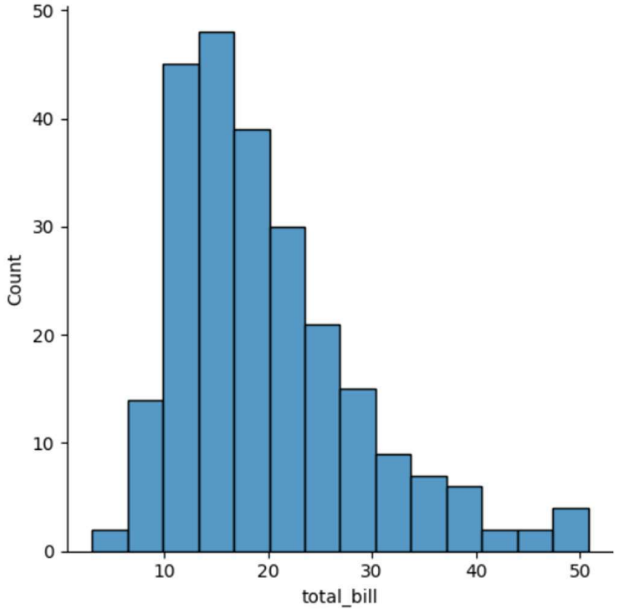
tips.sex.value_counts().plot(kind='bar')

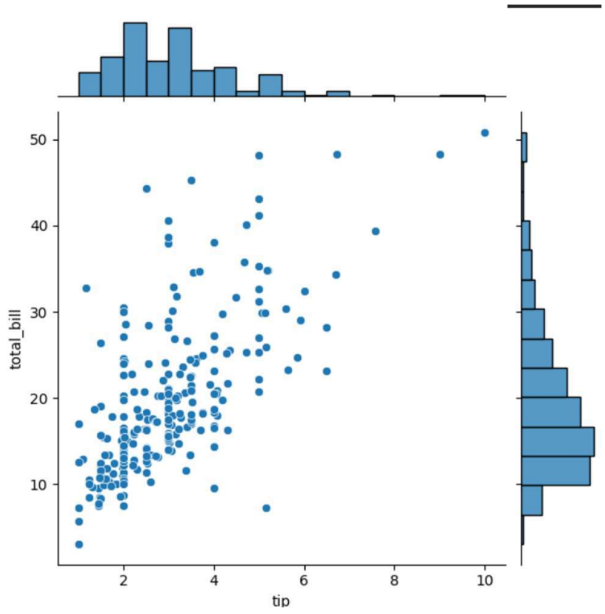
sns.countplot(tips[tips.time=='Dinner']['day'])
```

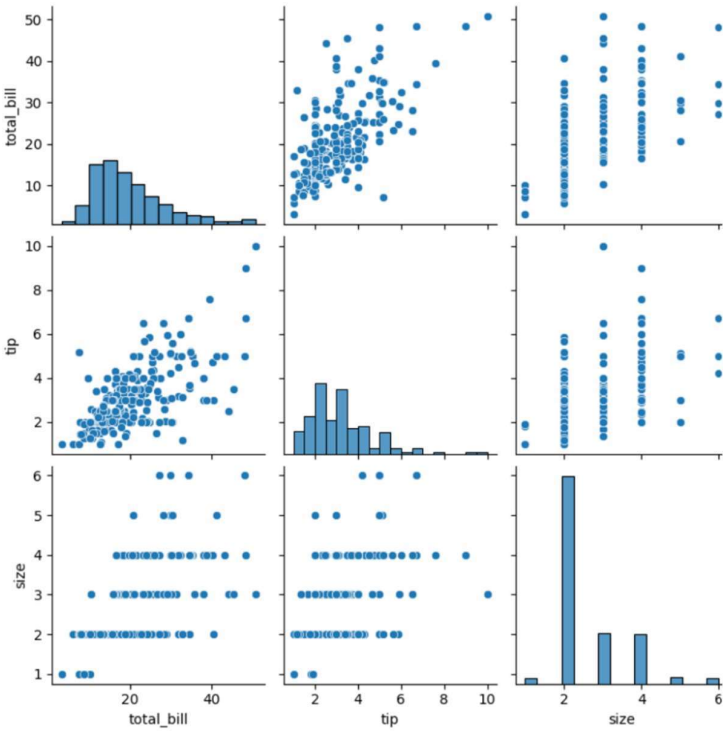


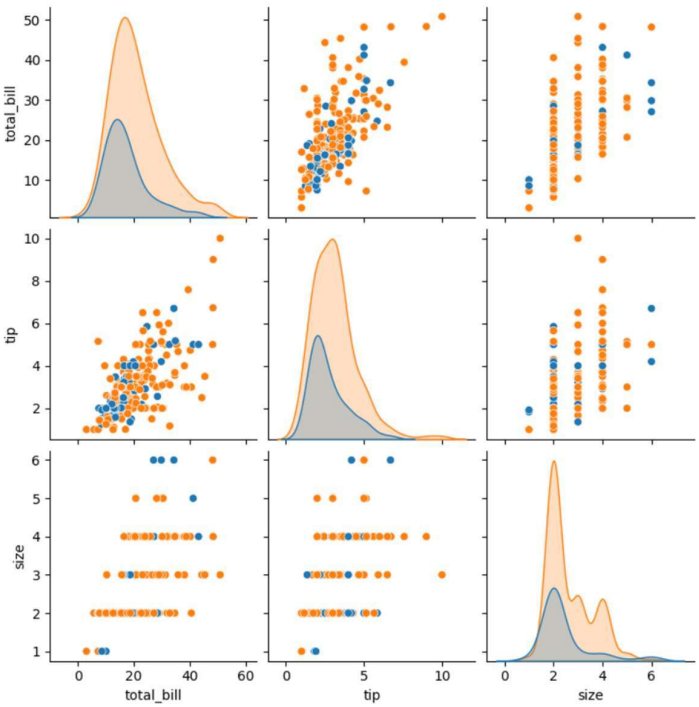


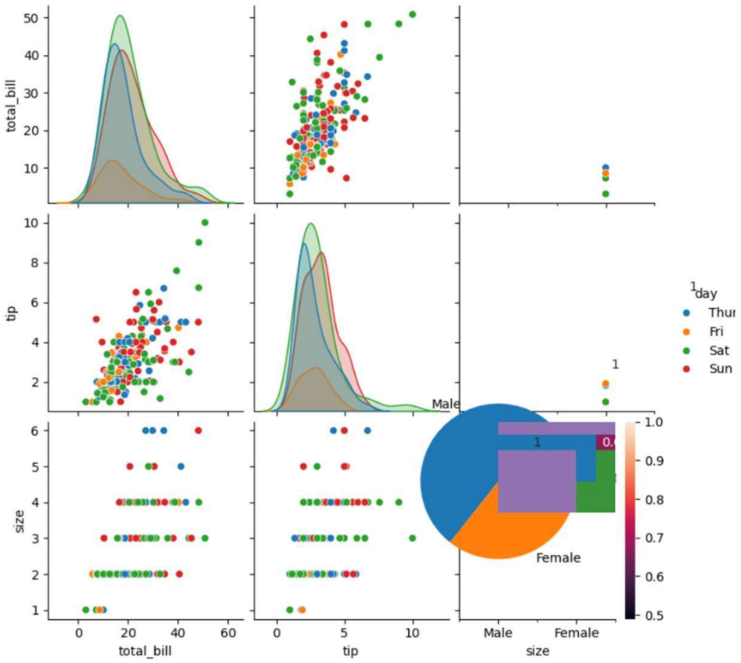

total_bill
tip
50
40
30
20
10
2
4
6
8
10


Count
total_bill
50
40
30
20
10
0
10
20
30
40
50


total_bill
tip
50
40
30
20
10
2
4
6
8
10


total_bill
tip
size
50
40
30
20
10
10
8
6
4
2
6
5
4
3
2
1
20
40
2
4
6
8
10
2
4
6


total_bill
tip
size
0
20
40
60
0
5
10
0
2
4
6
1
2
3
4
5
6
2
4
6
8
10
10
20
30
40
50



Result:

Hence the qualitative and quantitative EDA had been successfully studied

Exp No:8 Experiment to understand Linear Regression for a given data set.

CODE

```
import numpy as np
import pandas as pd
df=pd.read_csv('Salary_data.csv')
df.dropna(inplace=True)
df.info()
df.describe()
features=df.iloc[:,[0]].values
label=df.iloc[:,[1]].values
from sklearn.model_selection import train_test_split
x_train,x_test,y_train,y_test=train_test_split(features,label,test_size=
0.2,random_st
from sklearn.linear_model import LinearRegression
model=LinearRegression()
model.fit(x_train,y_train)
model.score(x_train,y_train)
model.score(x_test,y_test)
model.coef_
model.intercept_
import pickle
pickle.dump(model,open('SalaryPred.model','wb'))
```

```
model=pickle.load(open('SalaryPred.model','rb'))

yr_of_exp=float(input("Enter Years of Experience: "))

yr_of_exp_NP=np.array([[yr_of_exp]])

Salary=model.predict(yr_of_exp_NP)

print("Estimated Salary for {} years of experience is {}: "
.format(yr_of_exp,Salary)
```

OUTPUT

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 30 entries, 0 to 29
Data columns (total 2 columns):
 #   Column           Non-Null Count  Dtype
---  ------           --------------  -----
 0   YearsExperience  30 non-null     float64
 1   Salary           30 non-null     int64
dtypes: float64(1), int64(1)
memory usage: 612.0 bytes
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 30 entries, 0 to 29
Data columns (total 2 columns):
 #   Column           Non-Null Count  Dtype
---  ------           --------------  -----
 0   YearsExperience  30 non-null     float64
 1   Salary           30 non-null     int64
dtypes: float64(1), int64(1)
memory usage: 612.0 bytes
```

	YearsExperience	Salary
count	30.000000	30.000000
mean	5.313333	76003.000000
std	2.837888	27414.429785
min	1.100000	37731.000000
25%	3.200000	56720.750000
50%	4.700000	65237.000000
75%	7.700000	100544.750000
max	10.500000	122391.000000

RESULT: Hence Linear Regression for a given data set has been analysed.

Exp No:9 Experiment to understand Logistic Regression for a given data set.

CODE

```
import numpy as np

import pandas as pd

df=pd.read_csv('Social_Network_Ads.csv')

df.head()

features=df.iloc[:,[2,3]].values

label=df.iloc[:,4].values, features

from sklearn.model_selection import train_test_split

from sklearn.linear_model import LogisticRegression

from sklearn.model_selection import train_test_split

from sklearn.linear_model import LogisticRegression

x_train,x_test,y_train,y_test=train_test_split(features,label,test_size=
0.2,

finalModel=LogisticRegression()

finalModel.fit(x_train,y_train)

print(finalModel.score(x_train,y_train))

print(finalModel.score(x_test,y_test))

from sklearn.metrics import classification_report

print(classification_report(label,finalModel.predict(features)))
```

OUTPUT

	User ID	Gender	Age	EstimatedSalary	Purchased
0	15624510	Male	19	19000	0
1	15810944	Male	35	20000	0
2	15668575	Female	26	43000	0
3	15603246	Female	27	57000	0
4	15804002	Male	19	76000	0
...	...	...	...	...	...
395	15691863	Female	46	41000	1
396	15706071	Male	51	23000	1
397	15654296	Female	50	20000	1
398	15755018	Male	36	33000	0
399	15594041	Female	49	36000	1

	User ID	Gender	Age	EstimatedSalary	Purchased
0	15624510	Male	19	19000	0
1	15810944	Male	35	20000	0
2	15668575	Female	26	43000	0
3	15603246	Female	27	57000	0
4	15804002	Male	19	76000	0

```
array([0, 0, 0, 0, 0, 0, 0, 1, 0, 0, 0, 0, 0, 0, 0, 0, 1, 1, 1, 1, 1, 1,
       1, 1, 1, 1, 1, 1, 0, 0, 0, 1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
       0, 0, 0, 0, 1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 1, 0, 0,
       0, 0, 0, 0, 0, 0, 0, 0, 0, 1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 1, 0, 0,
       0, 0, 0, 0, 0, 0, 0, 0, 0, 1, 0, 0, 0, 0, 0, 1, 0, 0, 0, 0, 0, 0,
       0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
       0, 0, 0, 0, 0, 1, 0, 0, 0, 0, 0, 0, 0, 0, 1, 0, 0, 0, 0, 0, 0, 0,
       0, 0, 0, 0, 0, 1, 1, 0, 0, 0, 0, 0, 0, 0, 1, 0, 0, 0, 0, 0, 0, 0,
       0, 0, 0, 0, 0, 0, 1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
       0, 0, 0, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 1, 0, 0, 0, 1, 0, 0, 0, 1,
       0, 1, 1, 1, 0, 0, 1, 1, 0, 1, 1, 0, 1, 1, 0, 1, 0, 0, 0, 1, 1, 0,
       1, 1, 0, 1, 0, 1, 0, 1, 0, 0, 1, 1, 0, 1, 0, 0, 1, 1, 0, 1, 1, 0,
       1, 1, 0, 0, 1, 0, 0, 1, 1, 1, 1, 1, 0, 1, 1, 1, 1, 0, 1, 1, 0, 1,
       0, 1, 0, 1, 1, 1, 1, 0, 0, 0, 1, 1, 0, 1, 1, 1, 1, 1, 0, 0, 0, 1,
       1, 0, 0, 1, 0, 1, 0, 1, 1, 0, 1, 0, 1, 1, 0, 1, 1, 0, 0, 0, 1, 1,
       0, 1, 0, 0, 1, 0, 1, 0, 0, 1, 1, 0, 0, 1, 1, 0, 1, 1, 0, 0, 1, 0,
       1, 0, 1, 1, 1, 0, 1, 0, 1, 1, 1, 0, 1, 1, 1, 1, 0, 1, 1, 1, 0, 1,
       0, 1, 0, 0, 1, 1, 0, 1, 1, 1, 1, 1, 1, 0, 1, 1, 1, 1, 1, 1, 0, 1,
       1, 1, 0, 1], dtype=int64)
```

```
Test 0.6875 Train0.63125 Random State 3
Test 0.7375 Train0.61875 Random State 4
Test 0.6625 Train0.6375 Random State 5
Test 0.65 Train0.640625 Random State 6
Test 0.675 Train0.634375 Random State 7
Test 0.675 Train0.634375 Random State 8
Test 0.65 Train0.640625 Random State 10
Test 0.6625 Train0.6375 Random State 11
Test 0.7125 Train0.625 Random State 13
Test 0.675 Train0.634375 Random State 16
Test 0.7 Train0.628125 Random State 17
Test 0.7 Train0.628125 Random State 21
Test 0.65 Train0.640625 Random State 24
Test 0.6625 Train0.6375 Random State 25
Test 0.75 Train0.615625 Random State 26
Test 0.675 Train0.634375 Random State 27
Test 0.7 Train0.628125 Random State 28
Test 0.6875 Train0.63125 Random State 29
Test 0.6875 Train0.63125 Random State 31
```

	precision	recall	f1-score	support
0	0.85	0.93	0.89	257
1	0.84	0.71	0.77	143
accuracy			0.85	400
macro avg	0.85	0.82	0.83	400
weighted avg	0.85	0.85	0.85	400

RESULT Hence Logistic Regression for the given data set has been alanysed.

Exp No:10 Experiment to understand K-means clustering algorithm for a given data set.

CODE

```
import numpy as np

import pandas as pd

import matplotlib.pyplot as plt

import seaborn as sns

%matplotlib inline

df=pd.read_csv('Mall_Customers.csv')

df.info()

df.head()

sns.pairplot(df)

from sklearn.cluster import KMeans

model=KMeans(n_clusters=5)

model.fit(features)

KMeans(n_clusters=5)

Final=df.iloc[:,[3,4]]

Final['label']=model.predict(features)

Final.head()

sns.set_style("whitegrid")

sns.FacetGrid(Final,hue="label",height=8) \

.map(plt.scatter,"Annual Income (k$)", "Spending Score (1-100)") \

.add_legend();
```

```
plt.show()

features_el=df.iloc[:,[2,3,4]].values

from sklearn.cluster import KMeans

wcss=[]

for i in range(1,10):

model=KMeans(n_clusters=i)

model.fit(features_el)

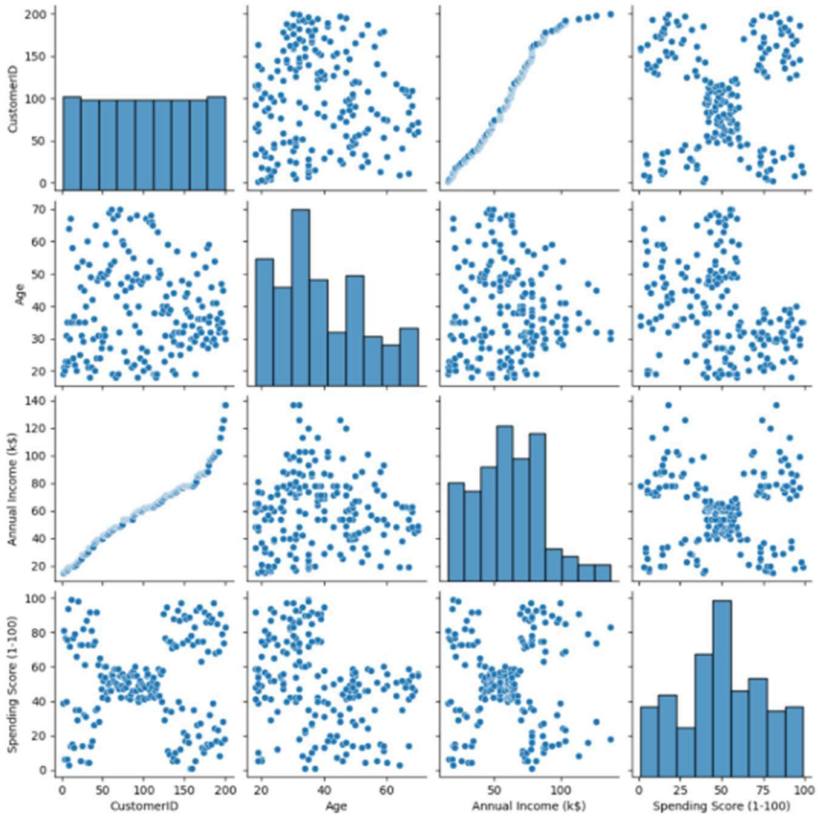
wcss.append(model.inertia_)

plt.plot(range(1,10),wcss)
```

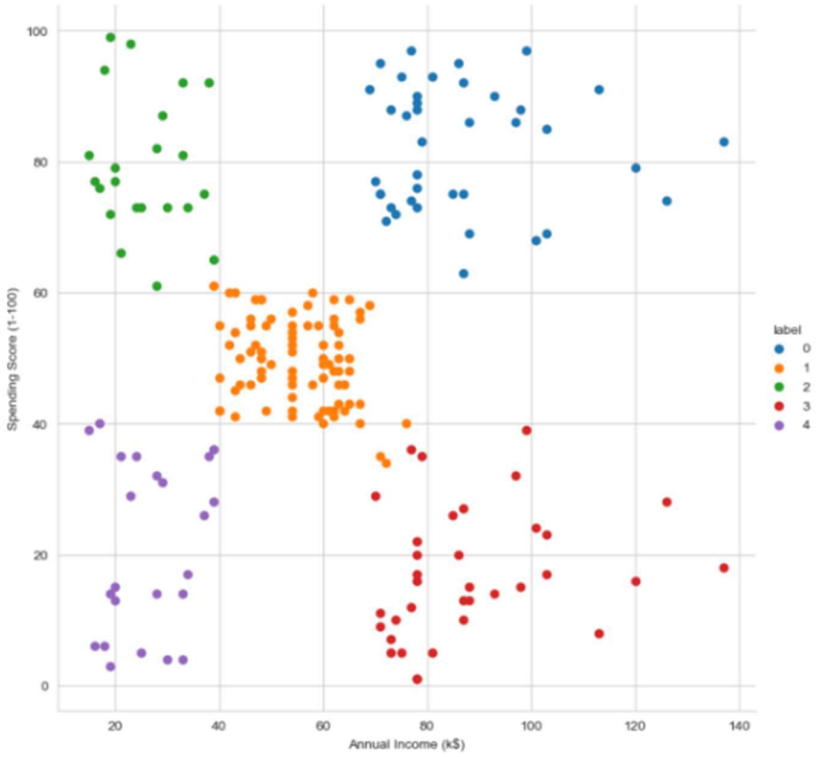
OUTPUT

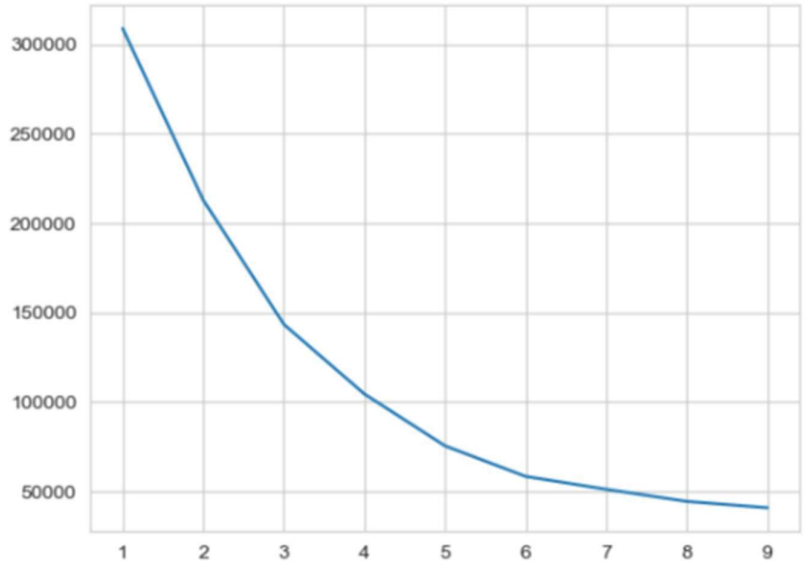
```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 200 entries, 0 to 199
Data columns (total 5 columns):
 #   Column                  Non-Null Count  Dtype
---  ------                  --------------  -----
 0   CustomerID              200 non-null    int64
 1   Gender                  200 non-null    object
 2   Age                     200 non-null    int64
 3   Annual Income (k$)      200 non-null    int64
 4   Spending Score (1-100)  200 non-null    int64
dtypes: int64(4), object(1)
memory usage: 7.9+ KB
```

	CustomerID	Gender	Age	Annual Income (k$)	Spending Score (1-100)
0	1	Male	19	15	39
1	2	Male	21	15	81
2	3	Female	20	16	6
3	4	Female	23	16	77
4	5	Female	31	17	40



	Annual Income (k$)	Spending Score (1-100)	label
0	15	39	4
1	15	81	2
2	16	6	4
3	16	77	2
4	17	40	4





RESULT Hence K-means clustering algorithm has been successfully studied.

Exp No:11 Experiment to understand KNN algorithm for a given data set

CODE

```
import numpy as np

import pandas as pd

df=pd.read_csv('Iris.csv')

df.info()

df.variety.value_counts()

df.head()

features=df.iloc[:,:-1].values

label=df.iloc[:,4].values

from sklearn.model_selection import train_test_split

from sklearn.neighbors import KNeighborsClassifier

xtrain,xtest,ytrain,ytest=train_test_split(features,label,test_size=.2,ra
ndo

model_KNN=KNeighborsClassifier(n_neighbors=5)

print(model_KNN.score(xtrain,ytrain))

print(model_KNN.score(xtest,ytest))

from sklearn.metrics import confusion_matrix

confusion_matrix(label,model_KNN.predict(features))

from sklearn.metrics import classification_report

print(classification_report(label,model_KNN.predict(features)))
```

OUTPUT

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 150 entries, 0 to 149
Data columns (total 5 columns):
 #   Column        Non-Null Count  Dtype
---  ------        --------------  -----
 0   sepal.length  150 non-null    float64
 1   sepal.width   150 non-null    float64
 2   petal.length  150 non-null    float64
 3   petal.width   150 non-null    float64
 4   variety       150 non-null    object
dtypes: float64(4), object(1)
memory usage: 6.0+ KB
```

	sepal.length	sepal.width	petal.length	petal.width	variety
0	5.1	3.5	1.4	0.2	Setosa
1	4.9	3.0	1.4	0.2	Setosa
2	4.7	3.2	1.3	0.2	Setosa
3	4.6	3.1	1.5	0.2	Setosa
4	5.0	3.6	1.4	0.2	Setosa

```
              precision    recall  f1-score   support

      Setosa       1.00      1.00      1.00        50
  Versicolor       0.96      0.94      0.95        50
   Virginica       0.94      0.96      0.95        50

    accuracy                           0.97       150
   macro avg       0.97      0.97      0.97       150
weighted avg       0.97      0.97      0.97       150
```

RESULT Hence KNN algorithm for the given data set has been analyzed.

Lab Experiment 12 Conduct Z test for the Data Given using Python

CODE

Copy code

```
import numpy as np

import scipy.stats as stats

sample_data = np.array([152, 148, 151, 149, 147, 153, 150, 148, 152, 149,151, 150, 149, 152, 151,

148, 150, 152, 149, 150,148, 153, 151, 150, 149, 152, 148, 151, 150, 153])

population_mean = 150

sample_mean = np.mean(sample_data)

sample_std = np.std(sample_data, ddof=1) # Using sample standard deviation

n = len(sample_data)

z_statistic = (sample_mean - population_mean) / (sample_std / np.sqrt(n)

p_value = 2 * (1 - stats.norm.cdf(np.abs(z_statistic))) # Two-tailed test

print(f"Sample Mean: {sample_mean:.2f}")

print(f"Z-Statistic: {z_statistic:.4f}")

print(f"P-Value: {p_value:.4f}")

alpha = 0.05

if p_value < alpha:

print("Reject the null hypothesis: The average weight is significantly different from 150 grams.")
```

```
else:
print("Fail to reject the null hypothesis: There is no significant
difference in average weight from
150 grams.")
```

OUTPUT:

```
Sample Mean: 150.20
Z-Statistic: 0.6406
P-Value: 0.5218
Fail to reject the null hypothesis: There is no significant difference in
average weight from 150 grams.
```

RESULT Hence Z test has been performed for the Data Given successfully.

Exp No:13.a Conduct an experiment to show data visualization using line plot

CODE:

```
import numpy as np

import pandas as pd

df = pd.DataFrame({

   'Country':
['India','Morocco','Germany','Morocco','Germany','India','Morocco','I
ndia','Germany','India'],

   'Age': [44, 27, 30, 38, 40, 35, np.nan, 48, 50, 37],

   'Salary': [72000, 48000, 54000, 61000, np.nan, 58000, 52000,
79000, 83000, 67000],

   'Purchased': ['No', 'Yes', 'No', 'No', 'Yes', 'Yes',

            'No', 'Yes', 'No', 'Yes']

})

print("Original Dataset:")

print(df)

print("\n\nDataFrame Information:")

print(df.info())

print("\n\nCountry Mode:")

print(df.Country.mode())

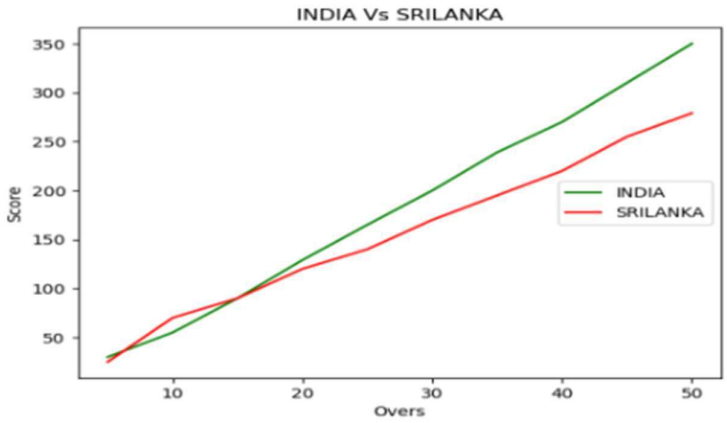
print("\n\nFirst Mode Value:")

print(df.Country.mode()[0])

df['Country'] = df['Country'].fillna(df.Country.mode()[0])
```

```
df['Age'] = df['Age'].fillna(df.Age.median())
df['Salary'] = df['Salary'].fillna(round(df.Salary.mean()))
print("\n\nDataset After Filling Missing Values:")
print(df)
print("\n\nOne Hot Encoding:")
print(pd.get_dummies(df.Country))
updated_dataset = pd.concat(
   [pd.get_dummies(df.Country), df.iloc[:, [1, 2, 3]]],
   axis=1
)
print("\n\nUpdated Dataset:")
print(updated_dataset)
print("\n\nDataFrame Information After Preprocessing:")
print(df.info())
updated_dataset['Purchased'] =
updated_dataset['Purchased'].replace(
   ['No', 'Yes'], [0, 1]
)
print("\n\nFinal Dataset:")
print(updated_dataset)
```

OUTPUT:



RESULT: data visualization using line plot has been successfully verified.

Exp No:13.b Conduct an experiment to show data visualization using bar chart

CODE

```
import matplotlib.pyplot as hscmark

import numpy as np

Names = ['SHREE', 'DEV', 'KEERTHI', 'PRIYA', 'SHAN', 'KUMARAN']

xaxis = np.arange(len(Names))

Percentage_hsc = [96, 91, 94, 75, 45, 81]

hscmark.bar(Names, Percentage_hsc)

hscmark.xticks(xaxis, Names, rotation=45)

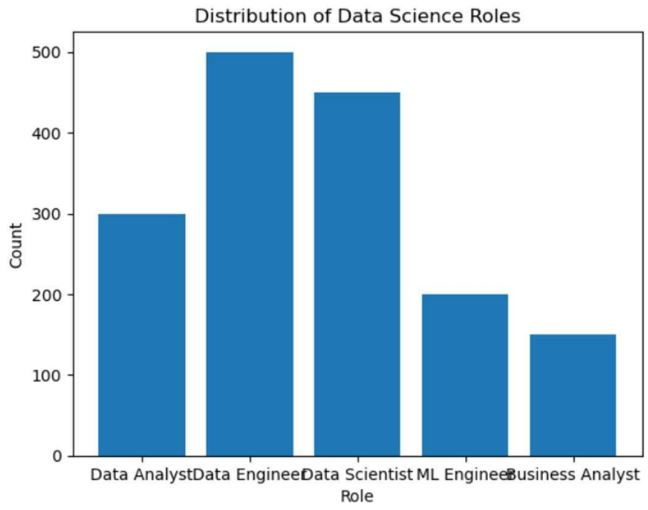
hscmark.xlabel("Names of Pupil")

hscmark.ylabel("Percentage")

hscmark.title("Comparison of HSC Percentage", fontsize=20,
color="green")

hscmark.show()
```

OUTPUT



RESULT: hence data visualization using bar chart has been represented successfully.

Exp No:13.c Conduct an experiment to show data visualization using pie chart

```
import matplotlib.pyplot as election

labels = ['CANDIDATE 1', 'CANDIDATE 2', 'CANDIDATE 3', 'CANDIDATE 4']

Votes = [315, 130, 245, 210]

colors = ['green', 'yellow', 'red', 'orange']

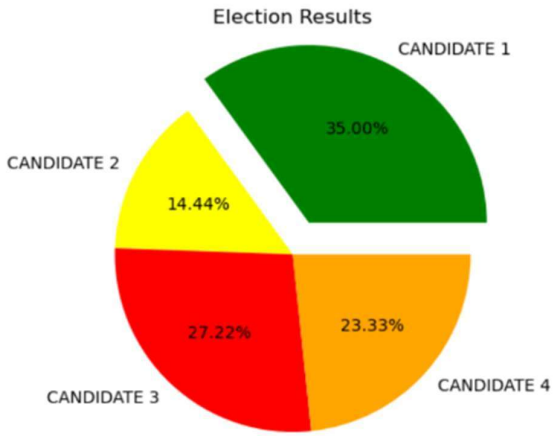
explode = (0.2, 0, 0, 0)

election.pie(Votes, labels=labels, colors=colors, explode=explode, autopct='%0.2f%%')

election.title('Election Results')

election.show()
```

OUTPUT



RESULT: data visualization using pie chart has been successfully verified.

Exp No:14 To Count the frequency of occurrence of a word in a body of text is often needed during text processing.

CODE:

```
import nltk

from nltk.tokenize import word_tokenize

from nltk.corpus import gutenberg

nltk.download('gutenberg')

nltk.download('punkt') # Ensure the punkt tokenizer models are also downloaded

sample = gutenberg.raw("austen-emma.txt") # Change to an existing text from the corpus

token = word_tokenize(sample)

wlist = []

for i in range(50):

wlist.append(token[i])

wordfreq = [wlist.count(w) for w in wlist]

print("Pairs\n" + str(list(zip(wlist, wordfreq))))
```

OUTPUT

```
Pairs
[('[', 1), ('Emma', 2), ('by', 1), ('Jane', 1), ('Austen', 1), ('1816',
1), (']', 1), ('VOLUME', 1), ('I', 2), ('CHAPTER', 1), ('I', 2), ('Emma
', 2), ('Woodhouse', 1), (',', 5), ('handsome', 1), (',', 5), ('clever'
, 1), (',', 5), ('and', 3), ('rich', 1), (',', 5), ('with', 2), ('a', 1
), ('comfortable', 1), ('home', 1), ('and', 3), ('happy', 1), ('disposi
tion', 1), (',', 5), ('seemed', 1), ('to', 1), ('unite', 1), ('some', 1
), ('of', 2), ('the', 2), ('best', 1), ('blessings', 1), ('of', 2), ('e
xistence', 1), (';', 1), ('and', 3), ('had', 1), ('lived', 1), ('nearly
', 1), ('twenty-one', 1), ('years', 1), ('in', 1), ('the', 2), ('world'
, 1), ('with', 2)]
```

RESULT: hence the frequency of occurrence of a word in a body of text is successfully verified.

Exp No:15 Data Collection and Initial Exploration

CODE:

```
import pandas as pd

diabetes_df = pd.read_csv("diabetes.csv")

print(diabetes_df.head())

print(db.info())

print(db.describe())

import matplot.pyplot as plt

import seaborn as sns

db.hist(bins=50, figsize=(20,15))

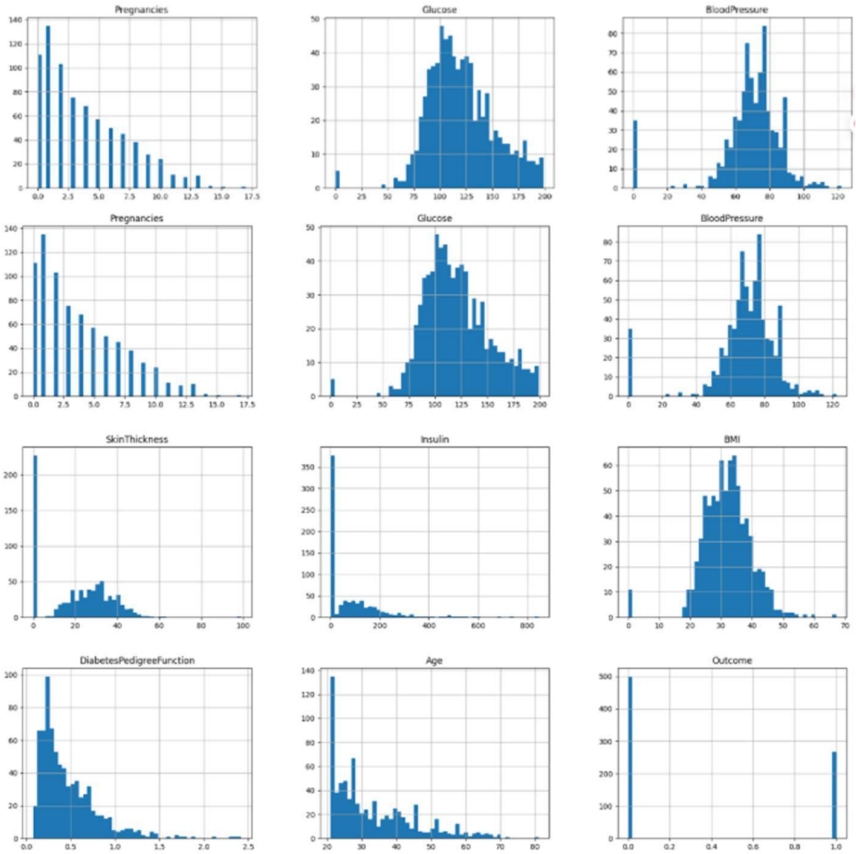
plt.show()

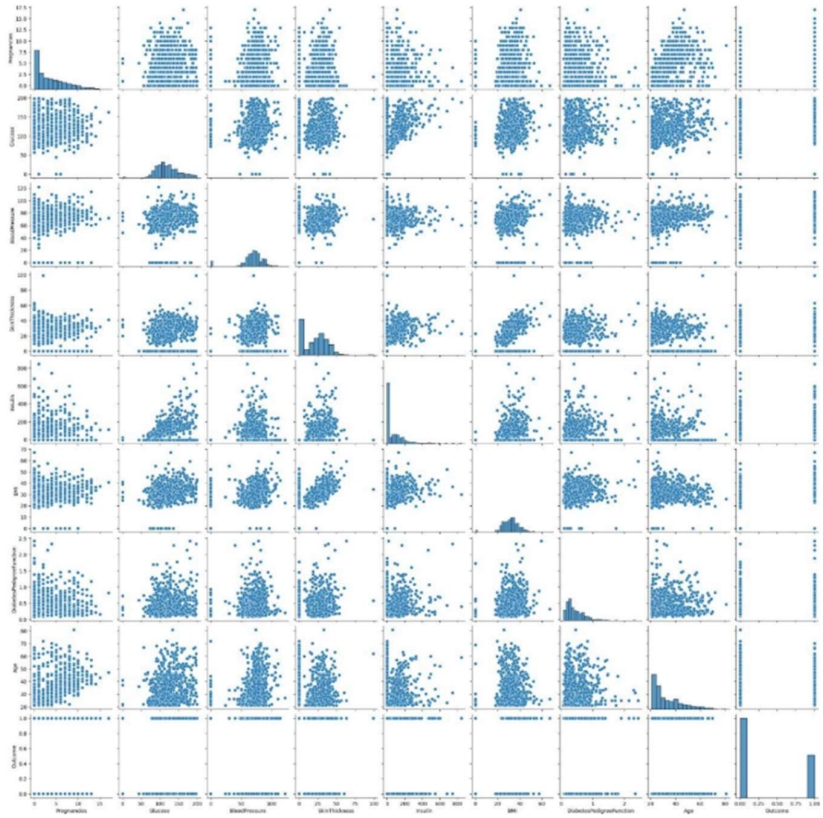
sns.pairplot(db)

plt.show()
```

OUTPUT

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 768 entries, 0 to 767
Data columns (total 9 columns):
 #   Column                    Non-Null Count  Dtype
---  ------                    --------------  -----
 0   Pregnancies               768 non-null    int64
 1   Glucose                   768 non-null    int64
 2   BloodPressure             768 non-null    int64
 3   SkinThickness             768 non-null    int64
 4   Insulin                   768 non-null    int64
 5   BMI                       768 non-null    float64
 6   DiabetesPedigreeFunction  768 non-null    float64
 7   Age                       768 non-null    int64
 8   Outcome                   768 non-null    int64
dtypes: float64(2), int64(7)
memory usage: 54.1 KB
None
       Pregnancies     Glucose  BloodPressure  SkinThickness     Insulin
count   768.000000  768.000000     768.000000     768.000000  768.000000
mean      3.845052  120.894531      69.105469      20.536458   79.799479
std       3.369578   31.972618      19.355807      15.952218  115.244002
min       0.000000    0.000000       0.000000       0.000000    0.000000
25%       1.000000   99.000000      62.000000       0.000000    0.000000
50%       3.000000  117.000000      72.000000      23.000000   30.500000
75%       6.000000  140.250000      80.000000      32.000000  127.250000
max      17.000000  199.000000     122.000000      99.000000  846.000000
```


Pregnancies
Glucose
BloodPressure
Pregnancies
Glucose
BloodPressure
SkinThickness
Insulin
BMI
DiabetesPedigreeFunction
Age
Outcome



RESULT: hence Data Collection and Initial Exploration has been studied.

Experiment 16: Time Series Analysis of Website Traffic

CODE:

```
import pandas as pd

import matplotlib.pyplot as plt

from statsmodels.tsa.seasonal import seasonal_decompose

data = pd.read_csv('website_traffic.csv', parse_dates=['Date'],
index_col='Date')

decomposition = seasonal_decompose(data['Page_Views'],
model='additive')

decomposition.plot()
```



```
plt.show()

data['Page_Views_MA'] =
data['Page_Views'].rolling(window=30).mean()

data[['Page_Views', 'Page_Views_MA']].plot()

plt.title('Page Views with Moving Average')

plt.show()

data['Page_Views_Z'] = (data['Page_Views'] -
data['Page_Views'].mean()) /

data['Page_Views'].std()

anomalies = data[data['Page_Views_Z'].abs() > 3]

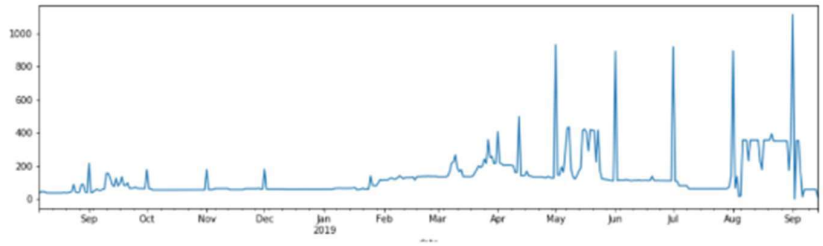
plt.plot(data.index, data['Page_Views'], label='Page Views')

plt.scatter(anomalies.index, anomalies['Page_Views'], color='red',
label='Anomalies')
```

```
plt.legend()

plt.show()
```

OUTPUT



RESULT: Hence Time Series Analysis of Website Traffic has been successfully studied.

Experiment 17:. Random Sampling and Sampling Distribution

CODE:

```
import numpy as np

import matplotlib.pyplot as plt

population_mean = 50

population_std = 10

population_size = 100000

population = np.random.normal(population_mean, population_std,
population_size)

sample_sizes = [30, 50, 100] # different sample sizes to consider

num_samples = 1000 # number of samples for each sample size
```



```
sample_means = {}

for size in sample_sizes:

sample_means[size] = []

for _ in range(num_samples):

sample = np.random.choice(population, size=size, replace=False)

sample_means[size].append(np.mean(sample))

plt.figure(figsize=(12, 8))

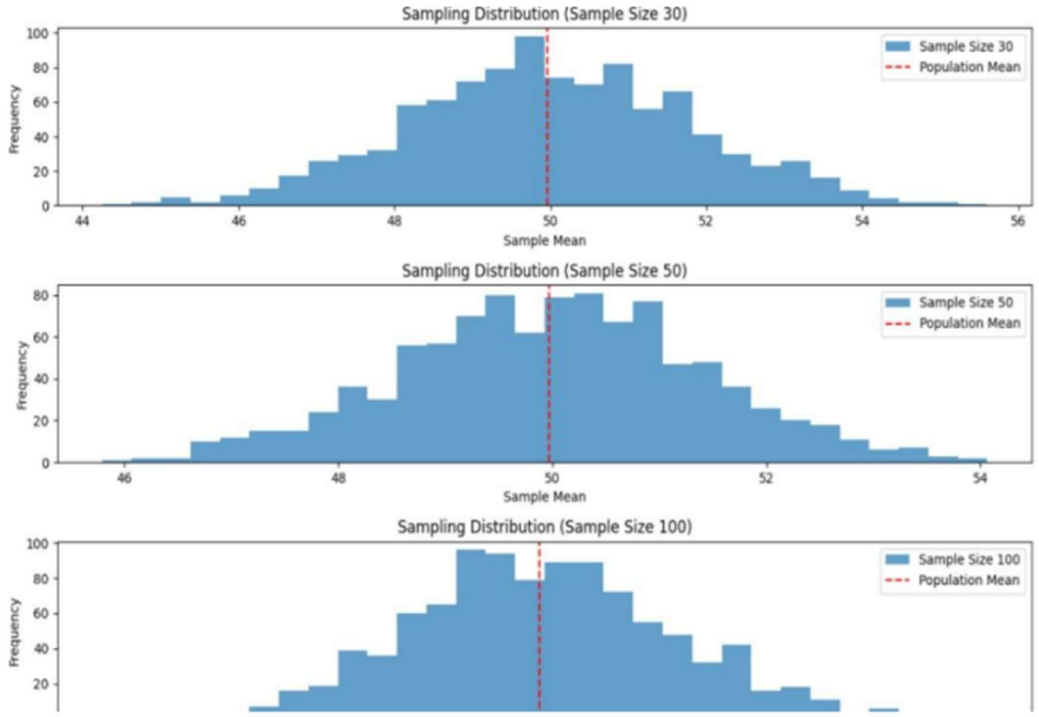
for i, size in enumerate(sample_sizes):

plt.subplot(len(sample_sizes), 1, i+1)

plt.hist(sample_means[size], bins=30, alpha=0.7, label=f'Sample Size
{size}')
```

```
plt.axvline(np.mean(population), color='red', linestyle='dashed',
linewidth=1.5, label='Population
Mean')
plt.title(f'Sampling Distribution (Sample Size {size})')
plt.xlabel('Sample Mean')
plt.ylabel('Frequency')
plt.legend()
plt.tight_layout()
plt.show()
```

OUTPUT



RESULT: Hence Random Sampling and Sampling Distribution hs been successfully verified.

Exp No:18 Experiment to understand Matplotlib library use cases in Data Science through visualization

CODE

```
import pandas as pd

import numpy as np

import seaborn as sns

import matplotlib.pyplot as plt

%matplotlib inline

data=pd.read_csv('/content/Iris_Dataset.csv')

data={ { ‘Id’ = (1,2,3,4,5) , ‘SepalLengthCm= (5.1, 4.9, 4.7, 4.6, 5.0),

        ‘SepalWidthCm= (3.5, 3.0, 3.2, 3.1, 3.6),

        ‘PetalLengthCm= (1.4, 1.4, 1.3, 1.5, 1.4),

        ‘PetalWidthCm = (5.1, 4.9, 4.7, 4.6, 5.0),

}

data.info()

data.describe()

data.value_counts('variety')

sns.countplot(x='variety',data=data,)

plt.show()

dummies=pd.get_dummies(data.variety)

FinalDataset=pd.concat([pd.get_dummies(data.variety),data.iloc[:,[0,1,2,3]]],axis=1)

FinalDataset.head()
```

```
sns.scatterplot(x='SepalLengthCm',y='SepalWidthCm',hue='variety',data=data,)

sns.scatterplot(x='PetalLengthCm',y='PetalWidthCm',hue='variety',data=data,)

sns.pairplot(data,hue='variety',height=3);

plt.show()

sns.FacetGrid(data,hue='variety',height=5).map(sns.histplot,'PetalLengthCm').add_legend();

plt.show();

sns.FacetGrid(data,hue='variety',height=5).map(sns.histplot,'PetalWidthCm').add_legend();

plt.show();

sns.FacetGrid(data,hue='variety',height=5).map(sns.histplot,'SepalLengthCm').add_legend();

plt.show();

sns.FacetGrid(data,hue='variety',height=5).map(sns.histplot,'SepalWidthCm').add_legend();

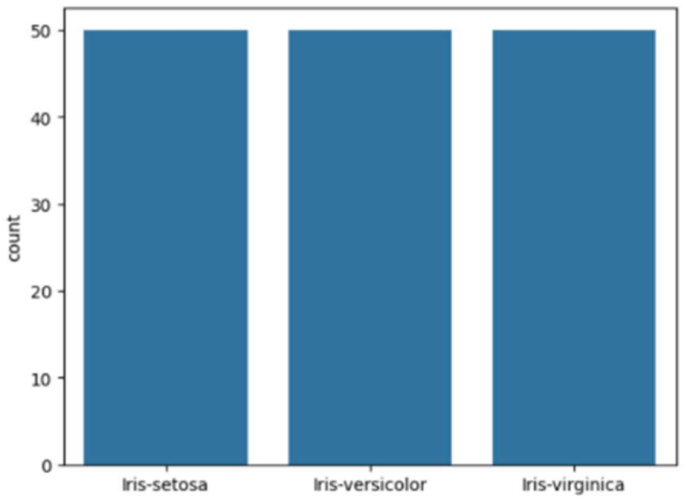
plt.show();
```

OUTPUT

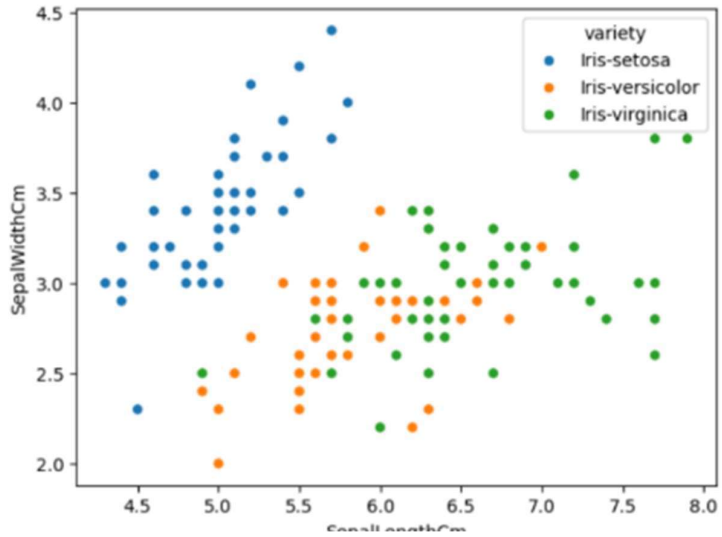
	Id	SepalLengthCm	SepalWidthCm	PetalLengthCm	PetalWidthCm	variety
0	1	5.1	3.5	1.4	0.2	Iris-setosa
1	2	4.9	3.0	1.4	0.2	Iris-setosa
2	3	4.7	3.2	1.3	0.2	Iris-setosa
3	4	4.6	3.1	1.5	0.2	Iris-setosa
4	5	5.0	3.6	1.4	0.2	Iris-setosa

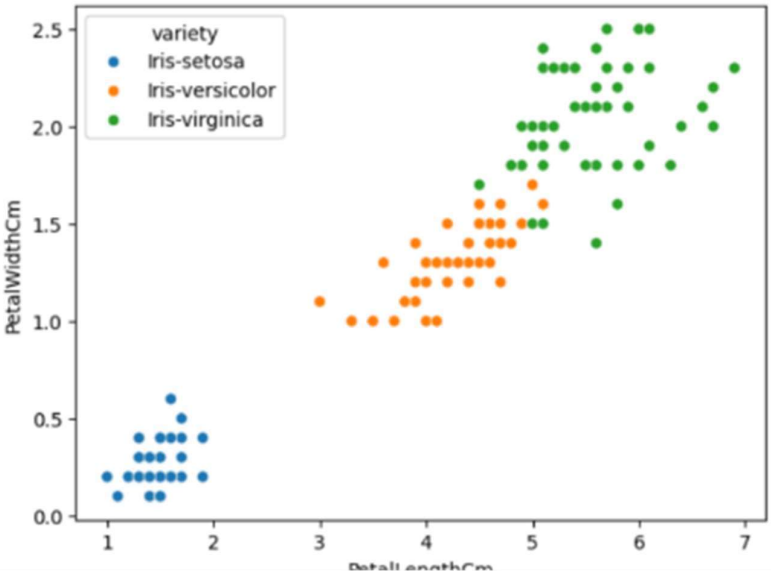
```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 150 entries, 0 to 149
Data columns (total 6 columns):
 #   Column         Non-Null Count  Dtype
---  ------         --------------  -----
 0   Id             150 non-null    int64
 1   SepalLengthCm  150 non-null    float64
 2   SepalWidthCm   150 non-null    float64
 3   PetalLengthCm  150 non-null    float64
 4   PetalWidthCm   150 non-null    float64
 5   variety        150 non-null    object
dtypes: float64(4), int64(1), object(1)
memory usage: 7.2+ KB
```

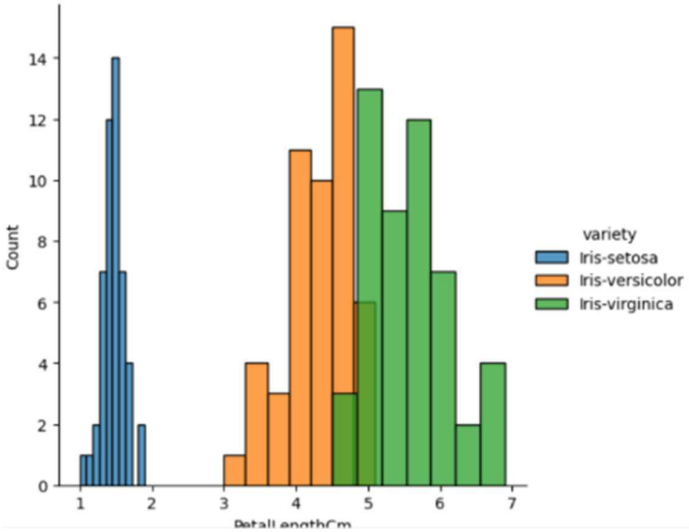
	Id	SepalLengthCm	SepalWidthCm	PetalLengthCm	PetalWidthCm
count	150.000000	150.000000	150.000000	150.000000	150.000000
mean	75.500000	5.843333	3.054000	3.758667	1.198667
std	43.445368	0.828066	0.433594	1.764420	0.763161
min	1.000000	4.300000	2.000000	1.000000	0.100000
25%	38.250000	5.100000	2.800000	1.600000	0.300000
50%	75.500000	5.800000	3.000000	4.350000	1.300000
75%	112.750000	6.400000	3.300000	5.100000	1.800000

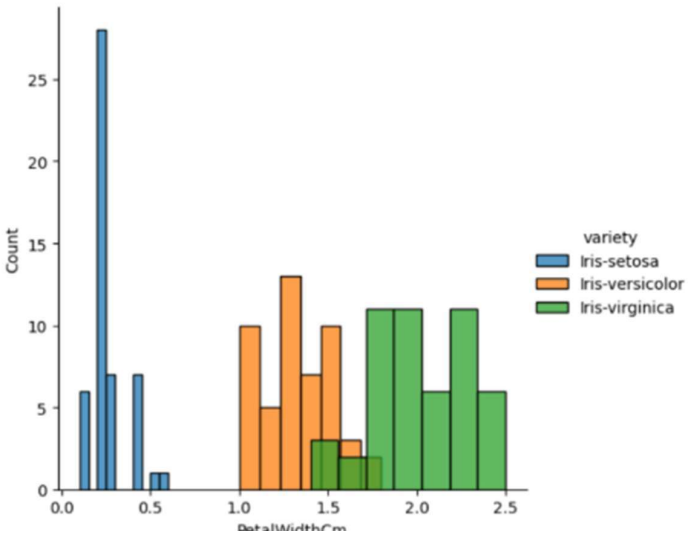


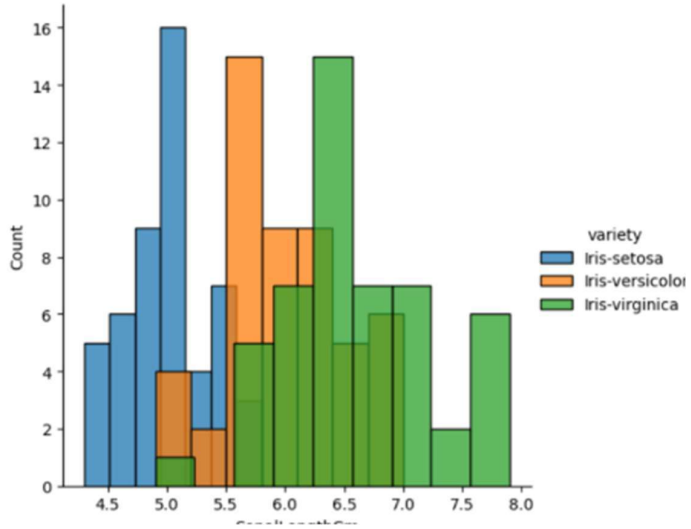
	Iris-setosa	Iris-versicolor	Iris-virginica	Id	SepalLengthCm	SepalWidthCm	PetalLengthCm
0	True	False	False	1	5.1	3.5	1.4
1	True	False	False	2	4.9	3.0	1.4
2	True	False	False	3	4.7	3.2	1.3
3	True	False	False	4	4.6	3.1	1.5

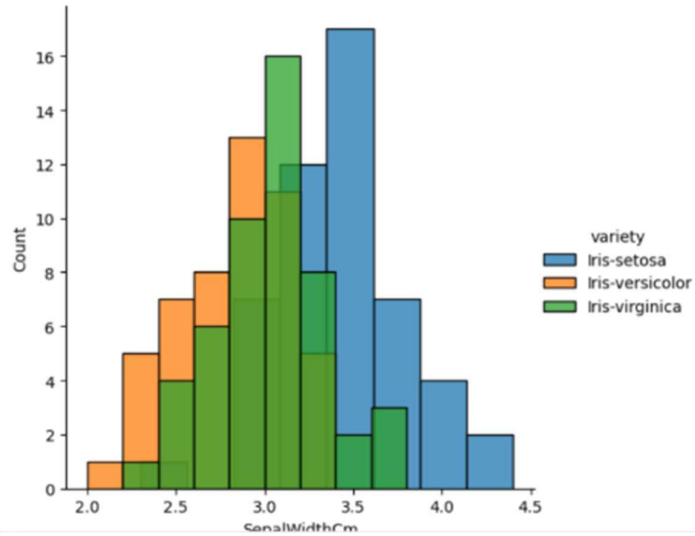

variety
Iris-setosa
Iris-versicolor
Iris-virginica
SepalWidthCm
4.5
4.0
3.5
3.0
2.5
2.0
4.5
5.0
5.5
6.0
6.5
7.0
7.5
8.0

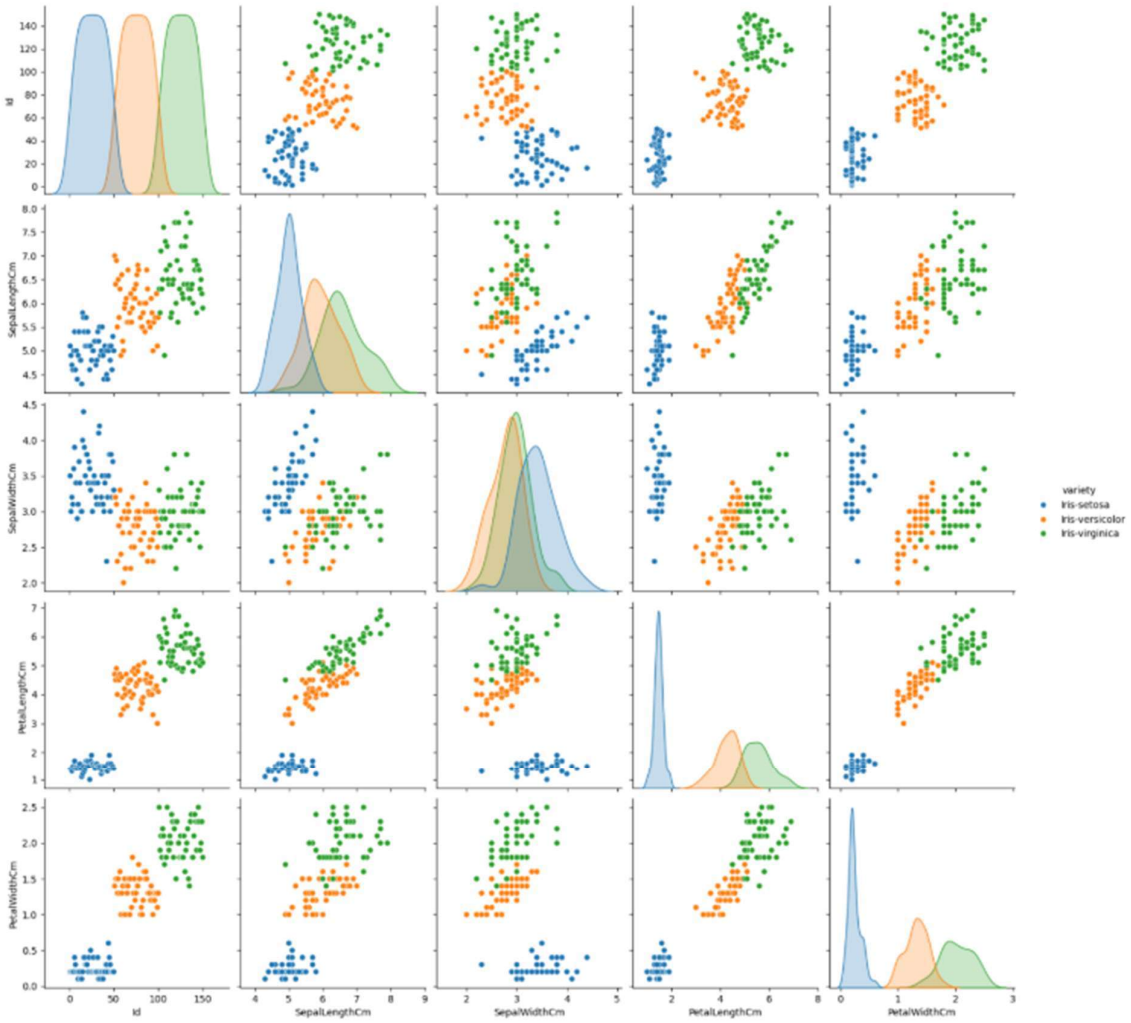

variety
Iris-setosa
Iris-versicolor
Iris-virginica
PetalWidthCm
2.5
2.0
1.5
1.0
0.5
0.0
1
2
3
4
5
6
7


Count
14
12
10
8
6
4
2
0
1
2
3
4
5
6
7
variety
Iris-setosa
Iris-versicolor
Iris-virginica


Count
PetalWidthCm
variety
Iris-setosa
Iris-versicolor
Iris-virginica


Count
variety
Iris-setosa
Iris-versicolor
Iris-virginica


Count
SepalWidthCm
variety
Iris-setosa
Iris-versicolor
Iris-virginica



RESULT: Hence understand Matplotlib library using cases in Data Science through visualization has been studied.

Exp No:19 Experiment to understand array function in Data science.

CODE

```
import numpy as np

array=np.random.randint(1,100,9)

array = [83, 25, 19, 47, 62, 15, 96, 39, 51]

np.sqrt(array)

array.ndim

new_array=array.reshape(3,3)

new_array

new_array.ndim

new_array.ravel()

newm=new_array.reshape(3,3)

newm[2,1:3]

newm[1:2,1:3]

new_array[0:3,0:0]

new_array[0:2,0:1]

new_array[0:3,0:1]

[47],

[96]])

new_array[1:3]
```

OUTPUT

array([83, 25, 19, 47, 62, 15, 96, 39, 51])

array([9.11043358, 5. , 4.35889894, 6.8556546 , 7.87400787,

3.87298335, 9.79795897, 6.244998 , 7.14142843])

array([[83, 25, 19],[47, 62, 15],[96, 39, 51]])

array([83, 25, 19, 47, 62, 15, 96, 39, 51])

array([[83, 25, 19], [47, 62, 15], [96, 39, 51]])

array([39, 51])

array([[62, 15]])

array([], shape=(3, 0), dtype=int64)

array([[83], [47]])

array([[83],[47],[96]])

RESULT: Hence array function in Data science has been successfully studied.

Exp No:20 Experiment to understand pandas library use cases in Data science.

CODE

```
import numpy as np

import pandas as pd

list=[[1,'Smith',50000],[2,'Jones',60000]]

df=pd.DataFrame(list)

df.columns=['Empd','Name','Salary']

df.columns=['Empd','Name','Salary']

df=pd.read_csv("/content/50_Startups.csv")

df.info()

df.head()

df.tail()

import numpy as np

import pandas as pd

df=pd.read_csv("/content/employee.csv")

df.head()

df.info()

df.salary

type(df.salary)

df.salary.mean()

df.salary.median()

df.salary.mode()
```

df.salary.var()

df.salary.std()

df.describe()

df.describe(include='all')

OUTPUT

	Empd	Name	Salary
0	1	Smith	50000
1	2	Jones	60000

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 50 entries, 0 to 49
Data columns (total 5 columns):
 #   Column           Non-Null Count  Dtype
---  ------           --------------  -----
 0   R&D Spend        50 non-null     float64
 1   Administration   50 non-null     float64
 2   Marketing Spend  50 non-null     float64
 3   State            50 non-null     object
 4   Profit           50 non-null     float64
dtypes: float64(4), object(1)
memory usage: 2.1+ KB
```

	R&D Spend	Administration	Marketing Spend	State	Profit
0	165349.20	136897.80	471784.10	New York	192261.83
1	162597.70	151377.59	443898.53	California	191792.06
2	153441.51	101145.55	407934.54	Florida	191050.39
3	144372.41	118671.85	383199.62	New York	182901.99

	emp id	name	salary
0	1	SREE VARSSINI K S	5000
1	2	SREEMATHI B	6000
2	3	SREYA G	7000
3	4	SREYASKARI MULLAPUDI	5000

	emp id	name	salary
2	3	SREYA G	7000
3	4	SREYASKARI MULLAPUDI	5000
4	5	SRI AKASH U G	8000
5	6	SRI HARSHAVARDHANAN R	3000

	emp id	salary
count	7.000000	7.000000
mean	4.000000	5714.285714
std	2.160247	1603.567451
min	1.000000	3000.000000
25%	2.500000	5000.000000
50%	4.000000	6000.000000
75%	5.500000	6500.000000

	emp id	name	salary
count	7.000000	7	7.000000
unique	NaN	6	NaN
top	NaN	SRI HARSHAVARDHANAN R	NaN
freq	NaN	2	NaN
mean	4.000000	NaN	5714.285714
std	2.160247	NaN	1603.567451
min	1.000000	NaN	3000.000000
25%	2.500000	NaN	5000.000000
50%	4.000000	NaN	6000.000000
75%	5.500000	NaN	6500.000000

RESULT: Hence pandas library using cases in Data science has been studied.